

Tensor models, tensor decomposition, and approximation

Vladimirov Eduard, group M05-304a

7 ноября 2024 г.

Аннотация

This essay explores fundamental tensor models, specifically focusing on CP and Tucker Decomposition, alongside Tensor Canonical Correlation Analysis (Tensor CCA). We introduce essential notation and operations, review core tensor decompositions, and provide insights into applications and theoretical underpinnings of tensor approximations in data science.

1 Introduction

Tensors, as multidimensional generalizations of matrices, are foundational in modern data science applications such as chemometrics, neuroscience, and machine learning. Tensor decompositions, including CP and Tucker Decomposition, extend concepts of linear algebra, such as the Singular Value Decomposition (SVD), to higher-dimensional arrays. Tensor CCA, a recent advancement, generalizes canonical correlation analysis to handle data from multiple views or sources.

2 Notation and Basic Operations

Following the conventions of multilinear algebra, tensors are denoted by boldface Euler script letters (e.g., $\mathcal{X}$), matrices by bold uppercase letters (e.g., $\mathbf{A}$), and vectors by bold lowercase letters (e.g., $\mathbf{a}$). A scalar entry within a tensor $\mathcal{X}$ is represented as $x_{i_1 i_2 \dots i_N}$.

2.1 Mode- n Product

Let $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ be an N -dimensional tensor. The *mode- n product* of $\mathcal{X}$ with a matrix $\mathbf{U} \in \mathbb{R}^{J \times I_n}$ is denoted by $\mathcal{X} \times_n \mathbf{U}$ and results in a new tensor of size $I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N$. Formally, it is defined as:

$$(\mathcal{X} \times_n \mathbf{U})_{i_1 \dots i_{n-1} j i_{n+1} \dots i_N} = \sum_{i_n=1}^{I_n} x_{i_1 i_2 \dots i_N} u_{j i_n}.$$

This operation transforms the n -th mode of $\mathcal{X}$ by multiplying it with the matrix $\mathbf{U}$, while leaving other modes unchanged.

2.2 Kronecker Product

The *Kronecker product* of two matrices $\mathbf{A} \in \mathbb{R}^{I \times J}$ and $\mathbf{B} \in \mathbb{R}^{K \times L}$, denoted by $\mathbf{A} \otimes \mathbf{B}$, is a block matrix of size $(I \cdot K) \times (J \cdot L)$. Formally, the Kronecker product is defined as:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} & \dots & a_{1J}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} & \dots & a_{2J}\mathbf{B} \\ \vdots & \vdots & \ddots & \vdots \\ a_{I1}\mathbf{B} & a_{I2}\mathbf{B} & \dots & a_{IJ}\mathbf{B} \end{bmatrix}.$$

The Kronecker product is widely used in tensor computations and is essential for expressing certain operations in tensor decompositions.

2.3 Khatri-Rao Product

The *Khatri-Rao product* of two matrices $\mathbf{A} \in \mathbb{R}^{I \times K}$ and $\mathbf{B} \in \mathbb{R}^{J \times K}$, denoted by $\mathbf{A} \odot \mathbf{B}$, is the column-wise Kronecker product, producing a matrix of size $(I \cdot J) \times K$. Formally, the Khatri-Rao product is defined as:

$$\mathbf{A} \odot \mathbf{B} = [\mathbf{a}_1 \otimes \mathbf{b}_1 \quad \mathbf{a}_2 \otimes \mathbf{b}_2 \quad \dots \quad \mathbf{a}_K \otimes \mathbf{b}_K],$$

where $\mathbf{a}_k$ and $\mathbf{b}_k$ are the k -th columns of $\mathbf{A}$ and $\mathbf{B}$, respectively. The Khatri-Rao product is often used in CP decomposition and tensor computations due to its role in forming compact expressions for multi-mode tensor products.

2.4 Mode- n Matricization (Unfolding)

To apply matrix operations to a tensor, we often need to convert the tensor into a matrix format, a process known as *matricization* or *unfolding*. The mode- n matricization of a tensor $\mathcal{X}$, denoted as $\mathbf{X}_{(n)}$, rearranges $\mathcal{X}$ into a matrix by aligning the n -th mode as rows and the other modes as columns. Specifically, for a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the mode- n matricization $\mathbf{X}_{(n)}$ is a matrix of size $I_n \times \left(\prod_{k \neq n} I_k\right)$, defined as:

$$\mathbf{X}_{(n)}(i_n, j) = x_{i_1 i_2 \dots i_N}$$

where $j = 1 + \sum_{k=1, k \neq n}^N (i_k - 1)J_k$ and $J_k = \prod_{m=1}^{k-1} I_m$ for $k < n$.

2.5 Example: Mode-3 Matricization of a 3rd-Order Tensor

For a third-order tensor $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$, the mode-3 matricization $\mathbf{X}_{(3)}$ would rearrange $\mathcal{X}$ into a matrix of size $K \times (I \cdot J)$, such that:

$$\mathbf{X}_{(1)}(i, j + (k - 1)J) = x_{ijk}.$$

$$\mathbf{X}_{(2)}(j, i + (k - 1)I) = x_{ijk}.$$

$$\mathbf{X}_{(3)}(k, i + (j - 1)I) = x_{ijk}.$$

The concept is easier to understand using an example. Let the frontal slices of $\mathcal{X} \in \mathbb{R}^{3 \times 4 \times 2}$ be

$$\mathcal{X}_1 = \begin{bmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{bmatrix}, \quad \mathcal{X}_2 = \begin{bmatrix} 13 & 16 & 19 & 22 \\ 14 & 17 & 20 & 23 \\ 15 & 18 & 21 & 24 \end{bmatrix}. \quad (1)$$

Then the three mode- n unfoldings are

$$\mathcal{X}_{(1)} = \begin{bmatrix} 1 & 4 & 7 & 10 & 13 & 16 & 19 & 22 \\ 2 & 5 & 8 & 11 & 14 & 17 & 20 & 23 \\ 3 & 6 & 9 & 12 & 15 & 18 & 21 & 24 \end{bmatrix},$$

$$\mathcal{X}_{(2)} = \begin{bmatrix} 1 & 2 & 3 & 13 & 14 & 15 \\ 4 & 5 & 6 & 16 & 17 & 18 \\ 7 & 8 & 9 & 19 & 20 & 21 \\ 10 & 11 & 12 & 22 & 23 & 24 \end{bmatrix},$$

$$\mathcal{X}_{(3)} = \begin{bmatrix} 1 & 2 & 3 & \dots & 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & \dots & 21 & 22 & 23 & 24 \end{bmatrix}.$$

2.6 Properties of Mode- n Product and Matricization

1. *Associativity*: The mode- n product is associative with respect to matrix multiplication:

$$(\mathcal{X} \times_n \mathbf{U}) \times_n \mathbf{V} = \mathcal{X} \times_n (\mathbf{V}\mathbf{U}).$$

2. *Commutativity*: For distinct modes in a series of multiplications, the order of the multiplication is irrelevant:

$$\mathcal{X} \times_m \mathbf{A} \times_n \mathbf{B} = \mathcal{X} \times_n \mathbf{B} \times_m \mathbf{A} \quad (m \neq n).$$

3. *Relation to Matricization*: The mode- n product can be expressed using mode- n matricization. Specifically, the mode- n product of $\mathcal{X}$ with $\mathbf{U}$ can be represented as:

$$(\mathcal{X} \times_n \mathbf{U})_{(n)} = \mathbf{U} \mathbf{X}_{(n)}.$$

4. *Inverse Operation*: If $\mathbf{U}$ is invertible, the mode- n product can be reversed by multiplying with $\mathbf{U}^{-1}$:

$$\mathcal{X} = (\mathcal{X} \times_n \mathbf{U}) \times_n \mathbf{U}^{-1}.$$

These operations are essential in tensor decompositions like CP and Tucker, where tensors are factorized through mode- n products and mode- n matricization enables matrix-based transformations.

3 CP Decomposition

The CANDECOMP/PARAFAC (Canonical decomposition + Parallel factor analysis or CP) decomposition, is one of the fundamental tensor decomposition methods, widely used in multilinear algebra, data analysis, and machine learning. It generalizes the concept of matrix rank decomposition to tensors by expressing a tensor as a sum of rank-one components. Given a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the CP decomposition seeks to approximate $\mathcal{X}$ as follows:

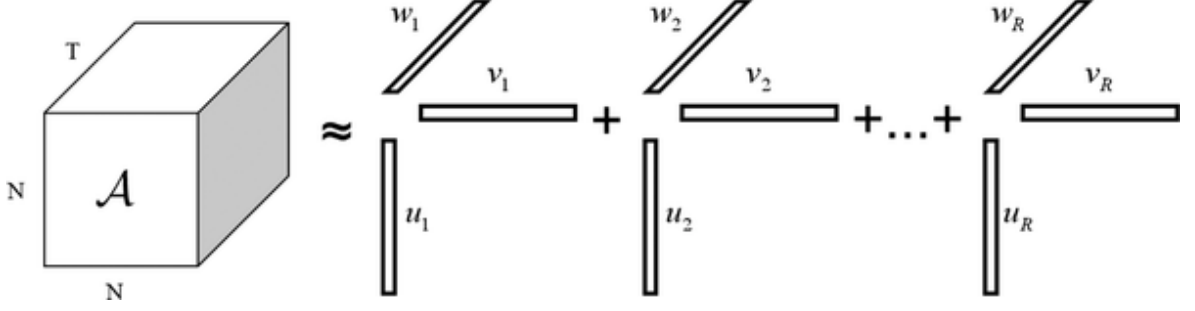


Рис. 1: CP Decomposition

$$\mathcal{X} \approx \sum_{r=1}^R \lambda_r \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)}, \quad (2)$$

where:

- R is the rank of the decomposition, representing the smallest number of rank-one tensors that approximate $\mathcal{X}$ with acceptable accuracy.
- λ_r are scalar weights for each rank-one component.
- $\mathbf{a}_r^{(n)} \in \mathbb{R}^{I_n}$ are vectors associated with the n -th mode for each rank- r component.
- $\circ$ denotes the outer product operation.

Thus, each term $\lambda_r \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)}$ is a rank-one tensor, and their sum provides an approximation of $\mathcal{X}$.

3.1 Rank of a Tensor and Uniqueness

The *rank* of a tensor $\mathcal{X}$, denoted as $\text{rank}(\mathcal{X})$, is defined as the minimum number of rank-one tensors required to represent $\mathcal{X}$ exactly (or approximately in the case of noise or incomplete data). Unlike matrices, tensors of higher order (order $N \geq 3$) do not have a unique rank in general, but under certain conditions, CP decomposition is unique up to scaling and permutation of components. This uniqueness is advantageous for applications where interpretability of components is essential.

3.2 Notation and Matrix Representation

To facilitate computations, the CP decomposition can also be represented in matrix form using mode- n unfoldings of $\mathcal{X}$. Let $\mathbf{X}_{(n)}$ denote the mode- n matricization of $\mathcal{X}$. Then, the CP decomposition can be represented as:

$$\mathbf{X}_{(n)} \approx \mathbf{A}^{(n)} \mathbf{\Lambda} \left(\mathbf{A}^{(N)} \odot \dots \odot \mathbf{A}^{(n+1)} \odot \mathbf{A}^{(n-1)} \odot \dots \odot \mathbf{A}^{(1)} \right)^T, \quad (3)$$

where:

- $\mathbf{A}^{(n)} = [\mathbf{a}_1^{(n)}, \mathbf{a}_2^{(n)}, \dots, \mathbf{a}_R^{(n)}] \in \mathbb{R}^{I_n \times R}$ is the factor matrix for the n -th mode.
- $\mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_R)$ is a diagonal matrix containing the weights λ_r .
- $\odot$ denotes the Khatri-Rao product, which is the column-wise Kronecker product of matrices.

This matrix form is commonly used in numerical methods to compute the CP decomposition efficiently.

3.3 Algorithms for Computing CP Decomposition

Several algorithms exist for computing the CP decomposition. The most popular methods include:

3.3.1 Alternating Least Squares (ALS)

The Alternating Least Squares (ALS) algorithm iteratively updates each factor matrix $\mathbf{A}^{(n)}$ while keeping other factors fixed. The objective is to minimize the difference between $\mathcal{X}$ and its approximation. The ALS update for each factor $\mathbf{A}^{(n)}$ is computed by solving a least-squares problem for $\mathbf{X}_{(n)}$ with respect to $\mathbf{A}^{(n)}$.

The ALS algorithm repeats this process for each mode until convergence, with the cost function given by:

$$\min_{\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}} \left\| \mathcal{X} - \sum_{r=1}^R \lambda_r \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)} \right\|^2.$$

Algorithm 1 CP-ALS Algorithm

procedure CP-ALS($\mathcal{X}, R$)

Initialize $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}$ for $n = 1, \dots, N$

repeat

for $n = 1, \dots, N$ **do**

$\mathbf{V} \leftarrow \mathbf{A}^{(1)T} \mathbf{A}^{(1)} * \dots * \mathbf{A}^{(n-1)T} \mathbf{A}^{(n-1)} * \mathbf{A}^{(n+1)T} \mathbf{A}^{(n+1)} * \dots *$
 $\mathbf{A}^{(N)T} \mathbf{A}^{(N)}$

$\mathbf{A}^{(n)} \leftarrow \mathcal{X}_{(n)}(\mathbf{A}^{(N)} \odot \dots \odot \mathbf{A}^{(n+1)} \odot \mathbf{A}^{(n-1)} \odot \dots \odot \mathbf{A}^{(1)}) \mathbf{V}^\dagger$

 Normalize columns of $\mathbf{A}^{(n)}$ (storing norms as λ)

end for

until fit ceases to improve or maximum iterations exhausted

return $\lambda, \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$

end procedure

3.3.2 Gradient-Based Optimization

In cases where ALS fails to converge or is inefficient, gradient-based optimization methods, such as stochastic gradient descent (SGD) or Adam, can be used to minimize the reconstruction error. These methods apply gradients directly to each factor matrix $\mathbf{A}^{(n)}$ and the weight vector $\boldsymbol{\lambda}$ to optimize the objective function.

3.3.3 Other Methods

Other methods include Tensor Power Iteration and methods based on higher-order orthogonal iteration (HOOI), though these are more common for Tucker decomposition.

3.4 Challenges in CP Decomposition

CP decomposition, while widely used, has challenges. Finding the correct rank R is difficult and often requires heuristics or cross-validation. Additionally, ALS can converge slowly or get stuck in local minima, and the decomposition may be sensitive to noise or missing values. Nevertheless, CP decomposition remains a powerful tool for analyzing multi-dimensional data in a structured and interpretable way.

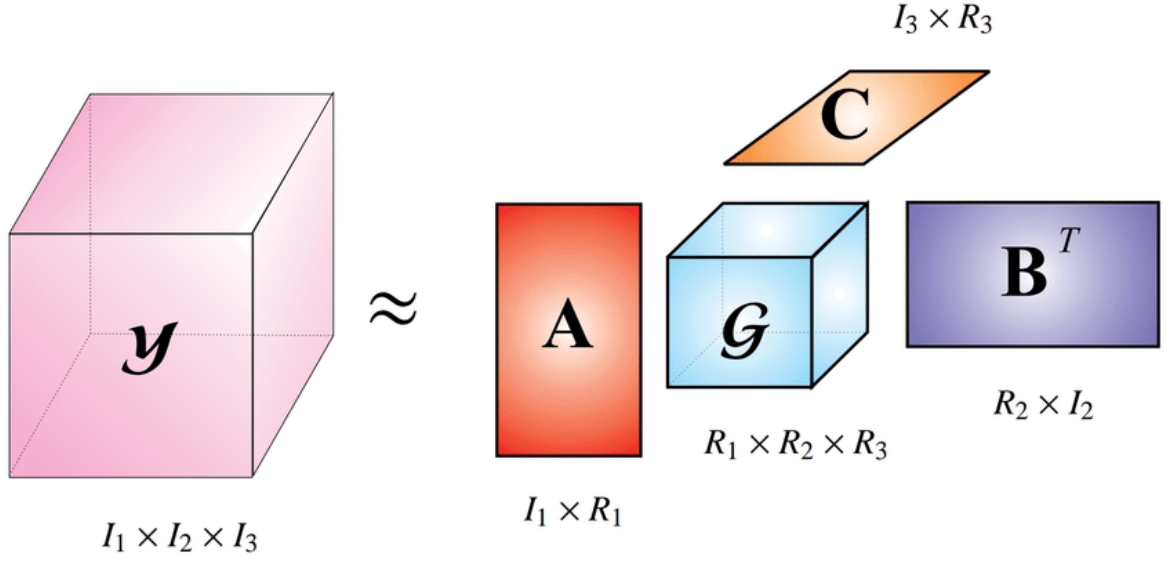


Рис. 2: Tucker Decomposition

4 Tucker Decomposition

The Tucker decomposition is a powerful tool in tensor decomposition that generalizes both Principal Component Analysis (PCA) and higher-order tensor decomposition methods. Also known as the *higher-order singular value decomposition* (HOSVD) or *multilinear SVD*, Tucker decomposition represents a tensor as a core tensor multiplied by factor matrices along each mode. This decomposition is particularly useful for dimensionality reduction and extracting latent factors in multi-way data.

4.1 Definition of Tucker Decomposition

Given an N -dimensional tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the Tucker decomposition approximates $\mathcal{X}$ as:

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \times_3 \dots \times_N \mathbf{A}^{(N)}, \quad (4)$$

where:

- $\mathcal{G} \in \mathbb{R}^{J_1 \times J_2 \times \dots \times J_N}$ is the *core tensor*, which captures the interactions between the components in each mode.
- $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times J_n}$ are the factor matrices, also called projection or loading matrices, which map the original space to a reduced space for each mode $n = 1, \dots, N$.

- $\times_n$ denotes the mode- n product.

In Tucker decomposition, the dimensions J_n of the core tensor $\mathcal{G}$ are usually smaller than the corresponding dimensions I_n of $\mathcal{X}$, making the decomposition an efficient and compressed representation of $\mathcal{X}$.

The matricized version of (4) is

$$\mathbf{X}_{(n)} \approx \mathbf{A}^{(n)} \mathcal{G}_{(n)} \left(\mathbf{A}^{(N)} \otimes \dots \otimes \mathbf{A}^{(n+1)} \otimes \mathbf{A}^{(n-1)} \otimes \dots \otimes \mathbf{A}^{(1)} \right)^T,$$

This matricized form is useful in computations as it simplifies the operations on the tensor to matrix multiplications, making it easier to implement and understand.

4.2 Interpretation of Tucker Decomposition

The Tucker decomposition can be interpreted as a form of dimensionality reduction in which each mode of the tensor is reduced by projecting it onto a lower-dimensional subspace. The core tensor $\mathcal{G}$ represents the compressed version of the original tensor, while the factor matrices $\mathbf{A}^{(n)}$ provide the basis for reconstructing the original data from the core tensor.

The core tensor can be thought of as capturing the "interactions" among the latent factors identified in each mode, while each factor matrix contains the principal components for its corresponding mode. Therefore, Tucker decomposition generalizes the concept of PCA to higher-order data structures.

4.3 Algorithms for Computing Tucker Decomposition

Various algorithms exist for computing the Tucker decomposition, with two prominent methods being:

4.4 HOSVD Algorithm

The Higher-Order Singular Value Decomposition (HOSVD) is a direct algorithm for Tucker decomposition. HOSVD performs SVD on each mode's matricization independently, providing an orthogonal basis for each mode. Here's the step-by-step procedure for HOSVD:

In the HOSVD algorithm, the tensor $\mathcal{X}$ is first matricized along each mode to form $\mathcal{X}_{(n)}$. Then, SVD is applied to each matricization independently, selecting the leading R_n singular vectors for each mode. This yields the

Algorithm 2 HOSVD

```
procedure HOSVD( $\mathcal{X}, R_1, R_2, \dots, R_N$ )  
  for  $n = 1, \dots, N$  do  
     $\mathbf{A}^{(n)} \leftarrow R_n$  leading left singular vectors of  $\mathcal{X}_{(n)}$   
  end for  
   $\mathcal{G} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)T} \times_2 \mathbf{A}^{(2)T} \dots \times_N \mathbf{A}^{(N)T}$   
  return  $\mathcal{G}, \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$   
end procedure
```

factor matrices $\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$, which are then used to compute the core tensor $\mathcal{G}$ through mode- n products.

4.5 HOOI Algorithm

The Higher-Order Orthogonal Iteration (HOOI) is an iterative refinement method that starts with an initial guess from HOSVD and iteratively improves the factor matrices to minimize reconstruction error. HOOI typically yields a more accurate approximation of the tensor at a higher computational cost.

Algorithm 3 HOOI

```
procedure HOOI( $\mathcal{X}, R_1, R_2, \dots, R_N$ )  
  Initialize  $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R_n}$  for  $n = 1, \dots, N$  using HOSVD  
  repeat  
    for  $n = 1, \dots, N$  do  
       $\mathcal{Y} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)T} \times \dots \times_{n-1} \mathbf{A}^{(n-1)T} \times_{n+1} \mathbf{A}^{(n+1)T} \dots \times_N \mathbf{A}^{(N)T}$   
       $\mathbf{A}^{(n)} \leftarrow R_n$  leading left singular vectors of  $\mathcal{Y}_{(n)}$   
    end for  
  until fit ceases to improve or maximum iterations exhausted  
   $\mathcal{G} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)T} \times_2 \mathbf{A}^{(2)T} \dots \times_N \mathbf{A}^{(N)T}$   
  return  $\mathcal{G}, \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$   
end procedure
```

In the HOOI algorithm, the initial factor matrices $\mathbf{A}^{(n)}$ are obtained using HOSVD. Each iteration then refines $\mathbf{A}^{(n)}$ by updating it to the R_n leading singular vectors of the mode- n unfolding $\mathcal{Y}_{(n)}$ of the partially reconstructed tensor $\mathcal{Y}$.

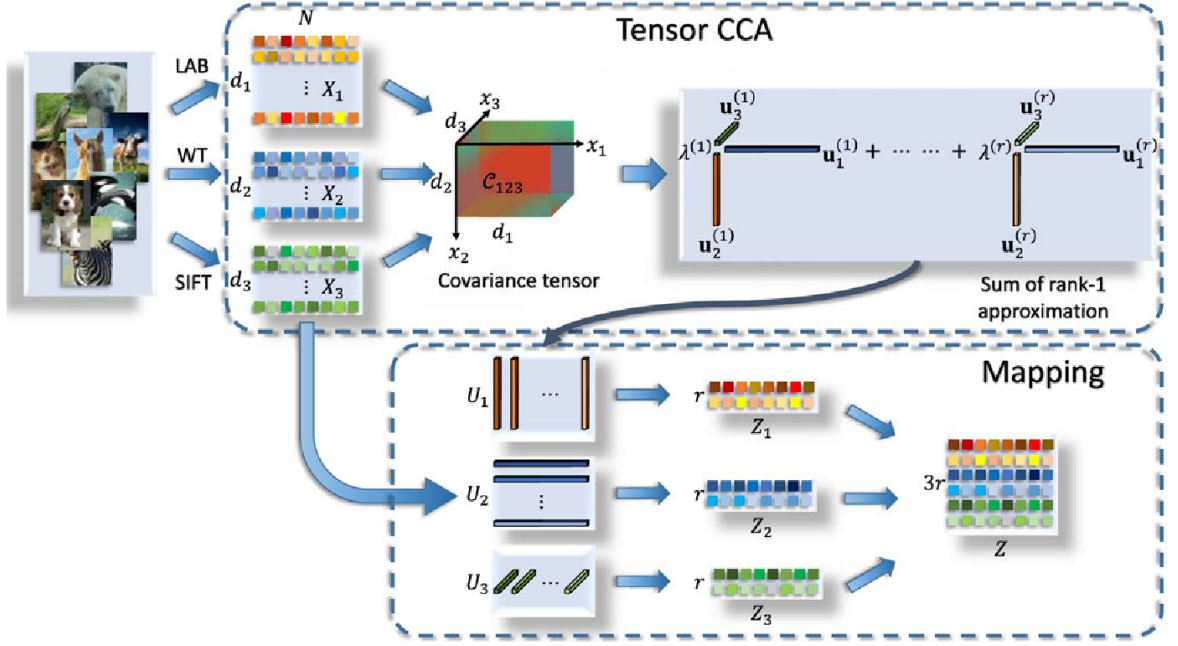


Рис. 3: Tensor CCA

5 Tensor Canonical Correlation Analysis (Tensor CCA)

Tensor Canonical Correlation Analysis (Tensor CCA) extends the traditional Canonical Correlation Analysis (CCA) to higher-dimensional tensor data, providing a method for analyzing relationships between two or more multi-dimensional arrays. In conventional CCA, the goal is to find linear transformations of two data matrices such that the transformed variables have maximum correlation. Tensor CCA generalizes this idea to tensors, enabling a more nuanced analysis of relationships in multi-way data, which is common in fields like neuroimaging, machine learning, and signal processing.

5.1 Problem notation

Given m views $\{X_p\}_{p=1}^m$ of N instances, where each $X_p = [x_{p1}, x_{p2}, \dots, x_{pN}] \in \mathbb{R}^{d_p \times N}$ is centered (i.e., has zero mean). The variance matrices are then defined as:

$$C_{pp} = \frac{1}{N} \sum_{n=1}^N x_{pn} x_{pn}^T, \quad p = 1, \dots, m,$$

and the covariance tensor among all views is calculated as:

$$C_{12\dots m} = \frac{1}{N} \sum_{n=1}^N x_{1n} \circ x_{2n} \circ \dots \circ x_{mn},$$

where C is a tensor of dimension $d_1 \times d_2 \times \dots \times d_m$. Following the objective of the traditional two-view CCA by Hardoon et al. (2004), the proposed tensor CCA seeks to maximize the correlation between the canonical variables $z_p = X_p^T h_p, p = 1, \dots, m$, where $\{h_p \in \mathbb{R}^{d_p \times 1}\}_{p=1}^m$ are usually called the canonical vectors. Therefore, the optimization problem is

$$\operatorname{argmax}_{\{h_p\}} \rho = \operatorname{corr}(z_1, z_2, \dots, z_m)$$

subject to $z_p^T z_p = 1, p = 1, \dots, m$. Here $\operatorname{corr}(z_1, z_2, \dots, z_m) = (z_1 \odot z_2 \odot \dots \odot z_m)^T e$ is the canonical correlation, and $\odot$ denotes the element-wise product, where $e \in \mathbb{R}^N$ is an all-ones vector. We can prove that it is equivalent to $C_{12\dots m} \times_1 h_1^T \times_2 h_2^T \dots \times_m h_m^T$, where $\times_p$ is the p -mode tensor-matrix product.

Theorem 1. The high order canonical correlation is given by

$$\rho = (z_1 \odot z_2 \odot \dots \odot z_m)^T e = C_{12\dots m} \times_1 h_1^T \times_2 h_2^T \dots \times_m h_m^T.$$

The proof is presented in the Appendix. By further considering that $X_p X_p^T = C_{pp}$ for $p = 1, \dots, m$, the problem becomes

$$\operatorname{argmax}_{\{h_p\}} \rho = C_{12\dots m} \times_1 h_1^T \times_2 h_2^T \dots \times_m h_m^T$$

subject to $h_p^T C_{pp} h_p = 1, p = 1, \dots, m$. To control model complexity, we add a regularization term to the constraints:

$$h_p^T (C_{pp} + \varepsilon I) h_p = 1, \quad p = 1, \dots, m,$$

where I is the identity matrix and ε is a nonnegative trade-off parameter. Let each $u_p = \tilde{C}_{pp}^{1/2} h_p$ and $\mathcal{M} = C_{12\dots m} \times_1 \tilde{C}_{11}^{-1/2} \times_2 \tilde{C}_{22}^{-1/2} \dots \times_m \tilde{C}_{mm}^{-1/2}$, so that we can reformulate the problem as

$$\operatorname{argmax}_{\{u_p\}} \rho = \mathcal{M} \times_1 u_1^T \times_2 u_2^T \dots \times_m u_m^T$$

subject to $u_p^T u_p = 1, \quad p = 1, \dots, m$.

5.2 Solutions

It has been presented in De Lathauwer et al. (2000b) that the problem is equivalent to finding the best rank-1 approximation of the tensor $\mathcal{M}$, i.e., if we define $\tilde{\mathcal{M}} = \rho u_1 \circ u_2 \circ \dots \circ u_m$, then the optimization problem becomes

$$\operatorname{argmin}_{\{u_p\}} \|\mathcal{M} - \tilde{\mathcal{M}}\|_F^2.$$

The solution can be obtained using the alternating least square (ALS) algorithm. Other algorithms, such as the high-order power method (HOPM) and the tensor power method, can also be applied here for optimization.

6 Conclusion

This essay presented CP and Tucker decompositions and introduced Tensor CCA for multi-view correlation analysis. These tensor models enable advanced dimensionality reduction and efficient data representation in multidimensional settings, supporting numerous applications in data-intensive fields.

Список литературы

- Kolda, Tamara G и Brett W Bader (2009). «Tensor decompositions and applications». В: *SIAM review* 51.3, с. 455—500.
- Luo, Yong и др. (2015). «Tensor canonical correlation analysis for multi-view dimension reduction». В: *IEEE transactions on Knowledge and Data Engineering* 27.11, с. 3111—3124.

Algorithm 4 Tensor Canonical Correlation Analysis (TCCA)

procedure TCCA($\{X_p\}_{p=1}^m, R$)

Given m views $\{X_p\}_{p=1}^m$ of N instances, where each $X_p \in \mathbb{R}^{d_p \times N}$ and is centered (i.e., has zero mean).

Compute the variance matrices for each view:

$$C_{pp} = \frac{1}{N} \sum_{n=1}^N x_{pn} x_{pn}^T, \quad p = 1, \dots, m.$$

Compute the covariance tensor among all views:

$$C_{12\dots m} = \frac{1}{N} \sum_{n=1}^N x_{1n} \circ x_{2n} \circ \dots \circ x_{mn}.$$

Define the optimization problem:

$$\operatorname{argmax}_{\{h_p\}} \rho = \operatorname{corr}(z_1, z_2, \dots, z_m) = (z_1 \odot z_2 \odot \dots \odot z_m)^T e$$

where $z_p = X_p^T h_p$, $p = 1, \dots, m$, and $\odot$ denotes the element-wise product.

Solve for ρ by iterating through the following steps:

- Update each canonical vector h_p to maximize the correlation ρ with constraints.
- Optionally, apply regularization to control model complexity:

$$h_p^T (C_{pp} + \varepsilon I) h_p = 1, \quad p = 1, \dots, m.$$

- Reformulate using transformed vectors u_p such that $h_p = \tilde{C}_{pp}^{1/2} u_p$.

Return the canonical correlation ρ and the projection vectors $\{u_p\}_{p=1}^m$.

end procedure
